

Name :- Rucha Nitin pawar

Branch :- AI&DS

Div :- B

Roll No:- 70

Practical Batch :- B4

Aim :- Implement and analyze various methods of studying correlation and regression including karl pearson's coefficient and simple linear regression.

1. What is correlation ? How is it different from regression?

Correlation is a statistical measure that shows the strength and direction of the relationship between two variables. It tells whether variables move together (positive), in opposite directions (negative), or are unrelated.

Regression, on the other hand, is used to model the relationship between variables and predict the value of one variable based on another.

Difference:

Correlation measures association only, while regression provides a predictive relationship. Correlation is symmetric (no dependent/independent variable), while regression is not. Correlation gives a single value (coefficient), while regression gives an equation.

2. What is simple linear regression ? Explain with suitable examples.

Simple linear regression is a statistical method used to model the relationship between two variables by fitting a straight line. One variable is independent (input) and the other is dependent (output).

It is expressed as: $y = a + bx$ where y is the dependent variable, x is the independent variable, a is the intercept, and b is the slope (rate of change).

Example: If we study the relationship between hours studied (x) and marks scored (y), simple linear regression can help predict marks based on study hours. For instance, more study hours usually lead to higher marks, so the line will have a positive slope.

Another example is predicting salary (y) based on years of experience (x). As experience increases, salary tends to increase.

3. How do you calculate pearson correlation in python ?

Pearson correlation in Python can be calculated using libraries like NumPy, Pandas, or SciPy.

Using Pandas: You can use the `corr()` function: Example: `import pandas as pd df.corr(method='pearson')`

Using NumPy: `import numpy as np np.corrcoef(x, y)`

Using SciPy: `from scipy.stats import pearsonr pearsonr(x, y)`

4. What is multicollinearity ? Does it affect simple regression ?

Multicollinearity occurs when two or more independent variables in a regression model are highly correlated with each other. This makes it difficult to determine the individual effect of each variable on the dependent variable.

It mainly affects multiple linear regression, not simple linear regression.

In simple linear regression, there is only one independent variable, so multicollinearity does not exist.

5. A dataset shows strong correlation but poor prediction accuracy. Why ?

A dataset can show strong correlation but still give poor prediction accuracy because correlation only measures the strength of relationship, not how well one variable can predict another.

Possible reasons:

The relationship may not be linear, but correlation only captures linear patterns well. There may be outliers affecting prediction accuracy. Important variables might be missing from the model. The data may have high noise or randomness. Overfitting or underfitting in the model.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import pearsonr
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```

# Function to print boxed text
def print_box(text):
    print("\n" + "-" * 50)
    print("OUTPUT".center(50))
    print("-" * 50)
    print(text)
    print("-" * 50)

# -----
# Step 1: Create Dataset
# -----
data = {
    "Hours": [1, 2, 3, 4, 5, 6, 7, 8],
    "Marks": [2, 4, 5, 4, 5, 7, 8, 9]
}

df = pd.DataFrame(data)

output = ""
output += "Dataset:\n" + str(df) + "\n\n"

# -----
# Step 2: Karl Pearson Correlation
# -----
corr, p_value = pearsonr(df["Hours"], df["Marks"])
output += f"Karl Pearson Correlation Coefficient: {corr}\n\n"

# -----
# Step 3: Simple Linear Regression
# -----
X = df[["Hours"]]
y = df["Marks"]

model = LinearRegression()
model.fit(X, y)

slope = model.coef_[0]
intercept = model.intercept_

output += "Regression Equation:\n"
output += f"y = {intercept:.2f} + {slope:.2f}x\n\n"

# -----
# Step 4: Prediction
# -----
y_pred = model.predict(X)

new_hours = np.array([[9]])
predicted_marks = model.predict(new_hours)

output += f"Predicted Marks for 9 hours study: {predicted_marks[0]}\n\n"

# -----
# Step 5: Model Evaluation
# -----
mse = mean_squared_error(y, y_pred)
r2 = r2_score(y, y_pred)

output += f"Mean Squared Error: {mse}\n"
output += f"R2 Score: {r2}\n\n"

# Print boxed output
print_box(output)

# -----
# Step 6: Visualization
# -----
plt.scatter(df["Hours"], df["Marks"], label="Actual Data")
plt.plot(df["Hours"], y_pred, label="Regression Line")
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.title("Correlation and Regression Analysis")
plt.legend()
plt.show()

```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but warnings.warn()
```

OUTPUT

Dataset:

	Hours	Marks
0	1	2
1	2	4
2	3	5
3	4	4
4	5	5
5	6	7
6	7	8
7	8	9

Karl Pearson Correlation Coefficient: 0.9511897312113418

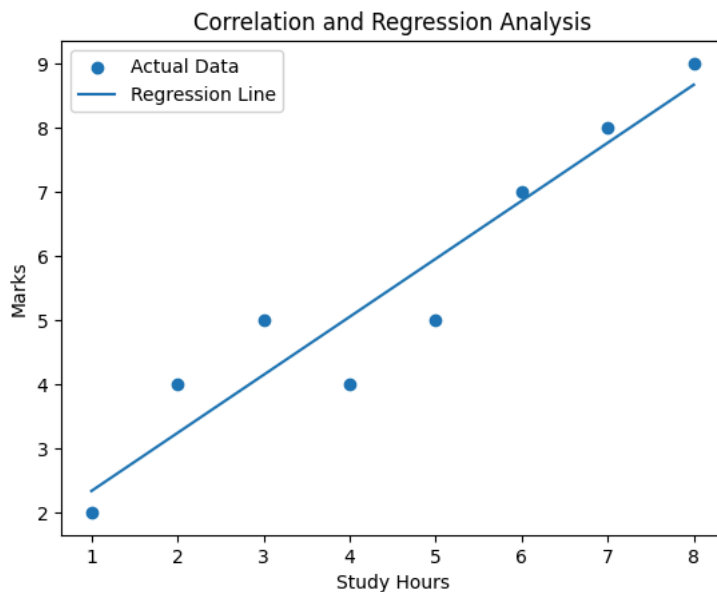
Regression Equation:

$y = 1.43 + 0.90x$

Predicted Marks for 9 hours study: 9.571428571428573

Mean Squared Error: 0.4523809523809522

R2 Score: 0.9047619047619048



Conclusion:

In this experiment, we implemented and analyzed correlation and simple linear regression using Python. Karl Pearson's coefficient helped measure the strength and direction of the relationship between variables, while simple linear regression was used to model and predict the relationship using a straight line.

The results showed how correlation indicates association between variables, and regression helps in making predictions based on that relationship. Overall, the experiment helped in understanding how correlation and regression are used together for statistical analysis and data prediction.

